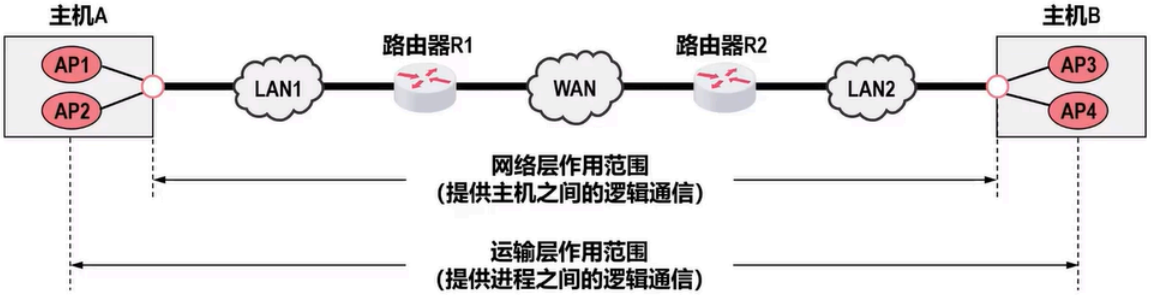
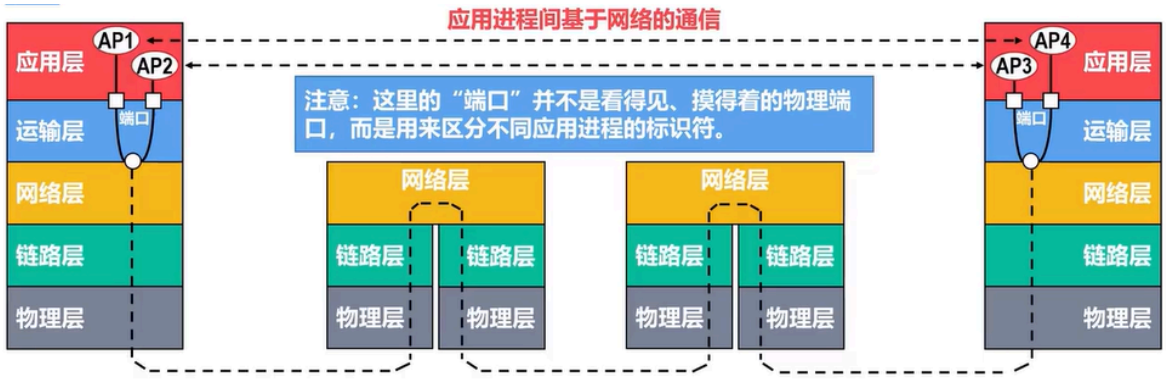
进程间基于网络的通信

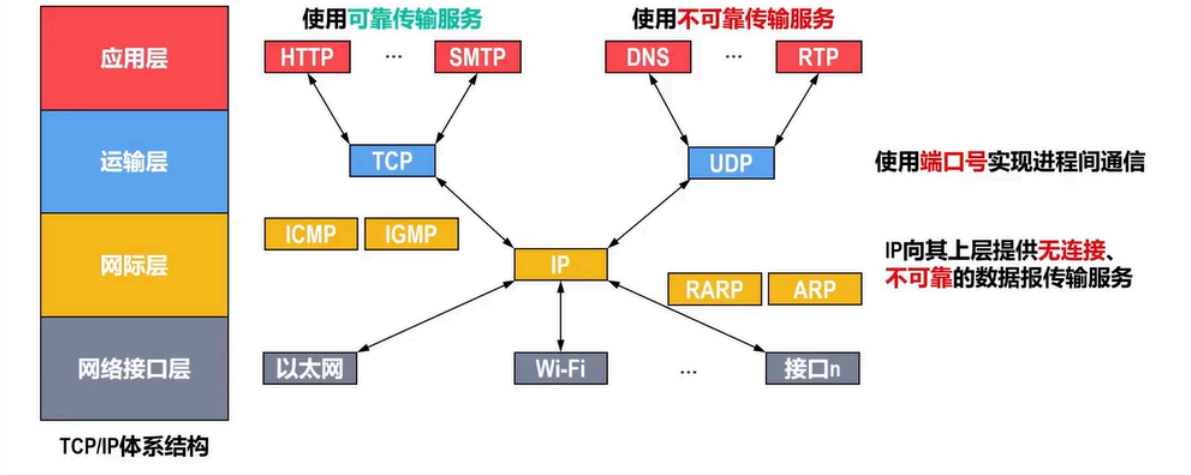
主要任务：为运行在不同主机上得应用进程提供直接的逻辑通信服务



向应用层实体屏蔽了下面网络核心的细节，好像是在两个运输层实体之间有一条端到端的逻辑通信信道



两个重要协议



TCP（传输控制协议）

- 面向连接的可靠数据传输服务
- 必须首先建立TCP连接（逻辑连接），结束后要释放
- 为了实现可靠传输有TCP连接管理、确认机制、超时重传、流量控制和拥塞控制等
- 实现复杂，报文段首部比较大，占用处理机资源比较多

UDP（用户数据报协议）

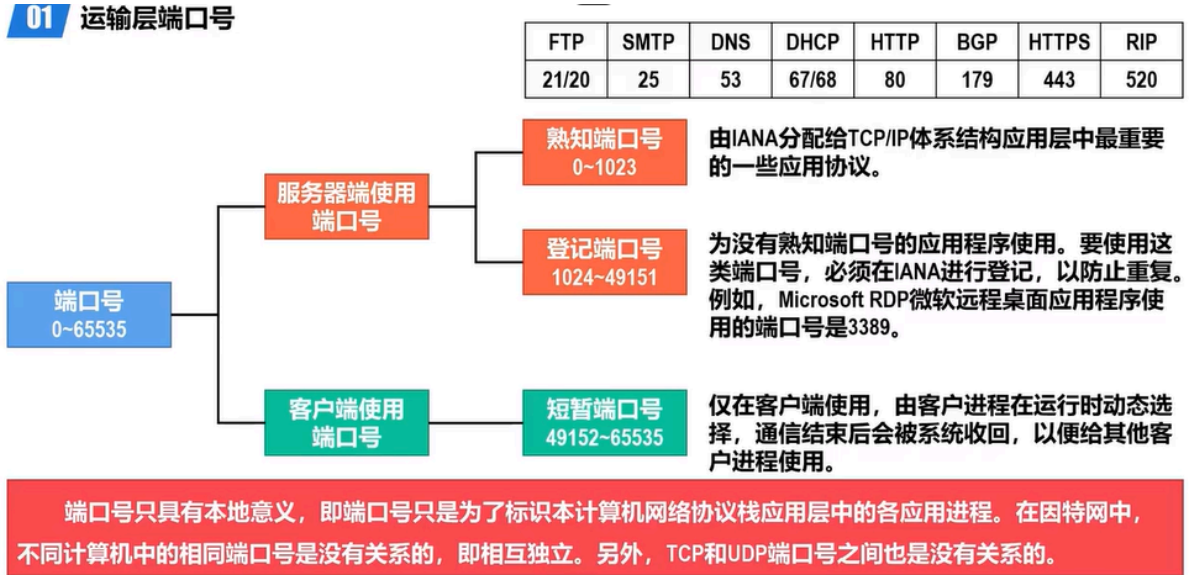
- 无连接不可靠
- 不需要建立连接

- 不需要使用实现可靠传输的各种机制
- 实现简单，首部比较小

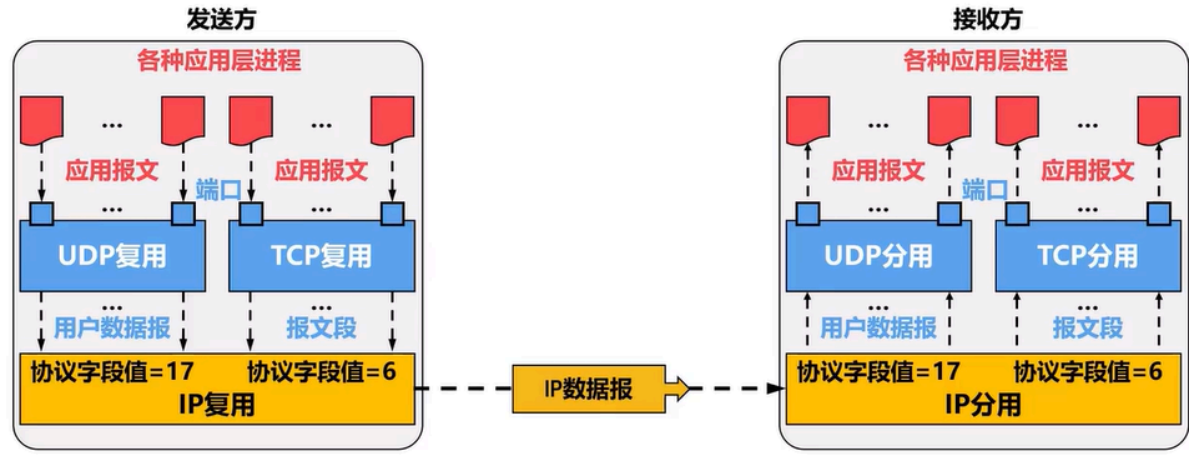
运输层端口号、复用与分用

端口号

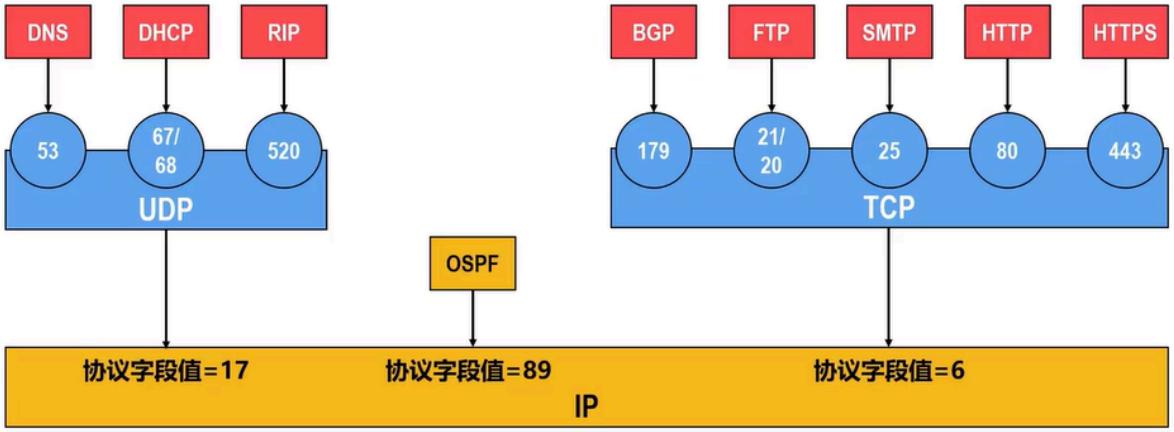
进程使用进程标识符（PID）标识，不同操作系统格式不同，因此使用端口号来标识不同的应用进程，长度为16比特，取值范围是0~65535



发送方的复用和接收方的分用



IP数据报协议字段的值代表使用了何种协议单元



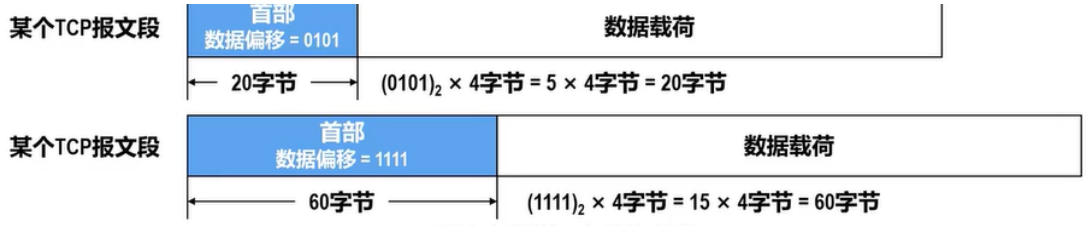
UDP和TCP对比

用户数据报协议UDP (User Datagram Protocol)	传输控制协议TCP (Transmission Control Protocol)
☐ 无连接	☐ 面向连接
☐ 支持“一对一”、“一对多”、“多对一”和“多对多”交互通信。	☐ 每一条TCP连接只能有两个端点EP，只能是一对一通信。
☐ 面向应用报文	☐ 面向字节流
☐ 尽最大努力交付，即不可靠；不使用流量控制和拥塞控制。	☐ 可靠传输，使用流量控制和拥塞控制。
☐ 首部开销小，仅8字节。	☐ 首部最小20字节，最大60字节。

TCP的首部格式



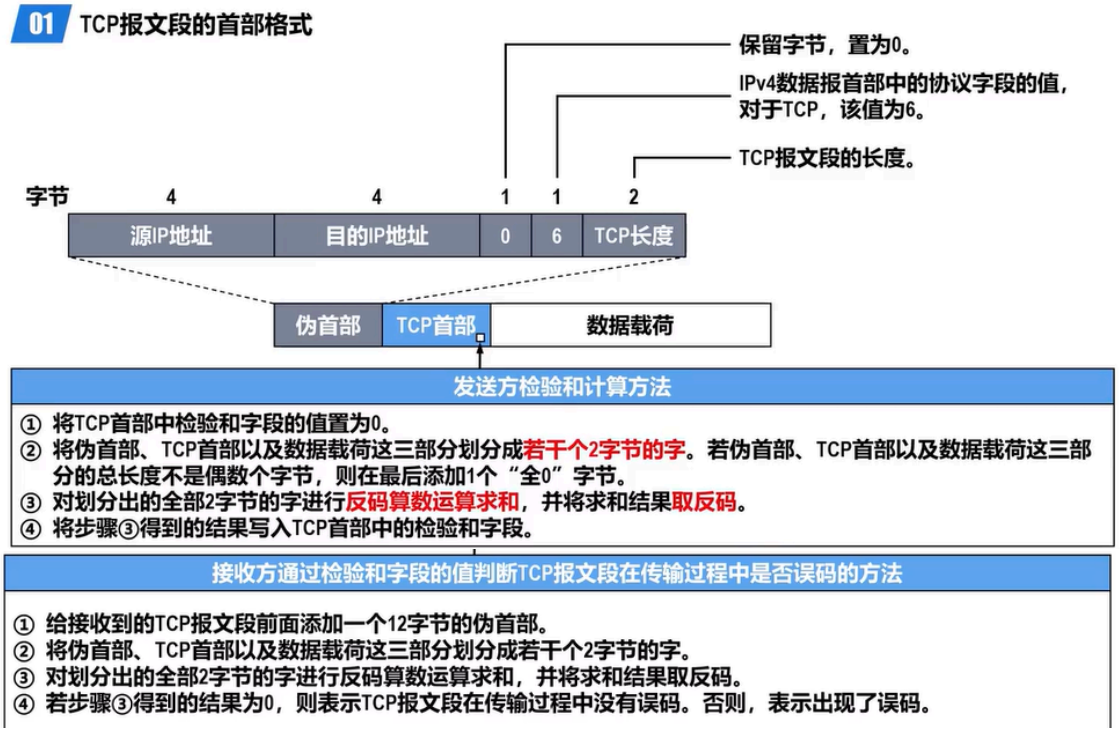
- 数据偏移：以4字节为单位，占4比特。指出了TCP报文段的首部长度



- 保留：占6比特，保留今后使用，应置为0

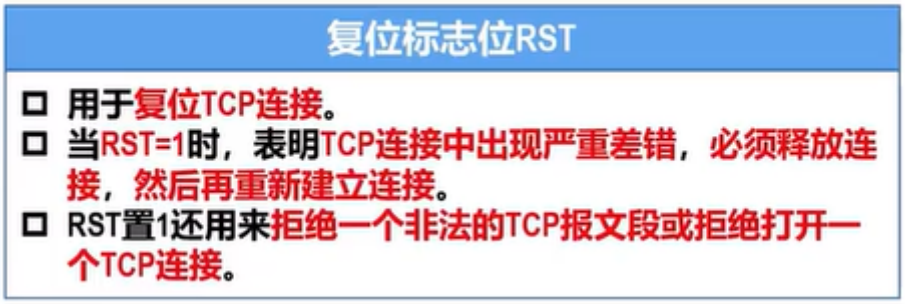
窗口
☐ 占16比特，该字段的取值**以字节为单位**。 ☐ 指出发送本报文段的一方的**接收窗口的大小**，即接收缓存的可用空间大小，这用来表征接收方的**接收能力**。 ☐ 在计算机网络中，经常**用接收方的接收能力的大小来控制发送方的数据发送量**，这就是所谓的**流量控制**。

- 检验和：占16比特，检验整个TCP报文段传输过程中是否出现误码









- 出于效率的考虑，TCP的发送方可能会延迟发送数据，而TCP的接收方可能会延迟向应用进程交付数据。这样可以一次处理更多的数据。
- 但是当两个应用进程进行交互式通信时，有时在一端的应用进程希望在键入一个命令后立即就能够收到对方的响应。在这种情况下，应用进程可以通知TCP使用推送（PUSH）操作。

推送标志位PSH

- 发送方TCP把PSH置1，并立即创建一个TCP报文段发送出去，而不需要积累到足够多的数据再发送。
- 接收方TCP收到PSH为1的TCP报文段，就尽快地交付给应用进程，而不再等到接收到足够多的数据才向上交付。

紧急标志位URG

- 当URG=1时，紧急指针字段有效。
- 当URG=0时，紧急指针字段无效。

紧急指针

- 占16比特，以字节为单位，用来指明紧急数据的长度。
- 当发送方有紧急数据时，可将紧急数据“插队”到发送缓存的最前面，并立刻封装到一个TCP报文段中进行发送。紧急指针会指出本报文段数据载荷部分包含了多长的紧急数据，紧急数据之后是普通数据。
- 接收方收到紧急标志位为1的TCP报文段，会按照紧急指针字段的值从报文段数据载荷中取出紧急数据并直接上交应用进程，而不必在接收缓存中排队。

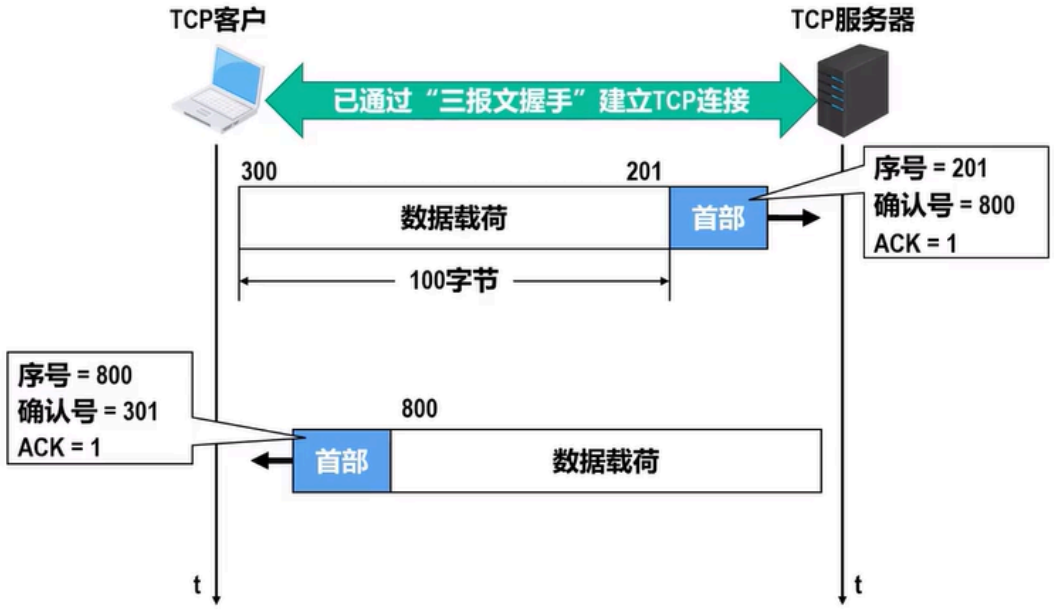
选项（长度可变，最大40字节）

- 最大报文段长度MSS选项：指出TCP报文段数据载荷部分的最大长度，而不是整个TCP报文段的长度。
- 窗口扩大选项：用来扩大窗口，提高吞吐率。
- 时间戳选项：
  - 用于计算往返时间RTT
  - 用于处理序号超范围的情况，又称为防止序号绕回PAWS。
- 选择确认选项：用来实现选择确认功能。

- 填充：若选择字段的长度加上20字节固定首部的长度不能被4字节整除时，需要填充相应数量的比特0，确保首部长度能被4字节整除

序号	占32比特，取值范围0~2^{32}-1。当序号增加到最后一个时，下一个序号又回到0。用来指出本TCP报文段数据载荷的第一个字节的序号。
确认号	占32比特，取值范围0~2^{32}-1。当确认号增加到最后一个时，下一个确认号又回到0。用来指出期望收到对方下一个TCP报文段的数据载荷的第一个字节的序号，同时也是对之前收到的所有数据的确认。
确认标志位ACK	只有当ACK取值为1时，确认号字段才有效。ACK取值为0时，确认号字段无效。 TCP规定：在TCP连接建立后，所有传送的TCP报文段都必须把ACK置1。

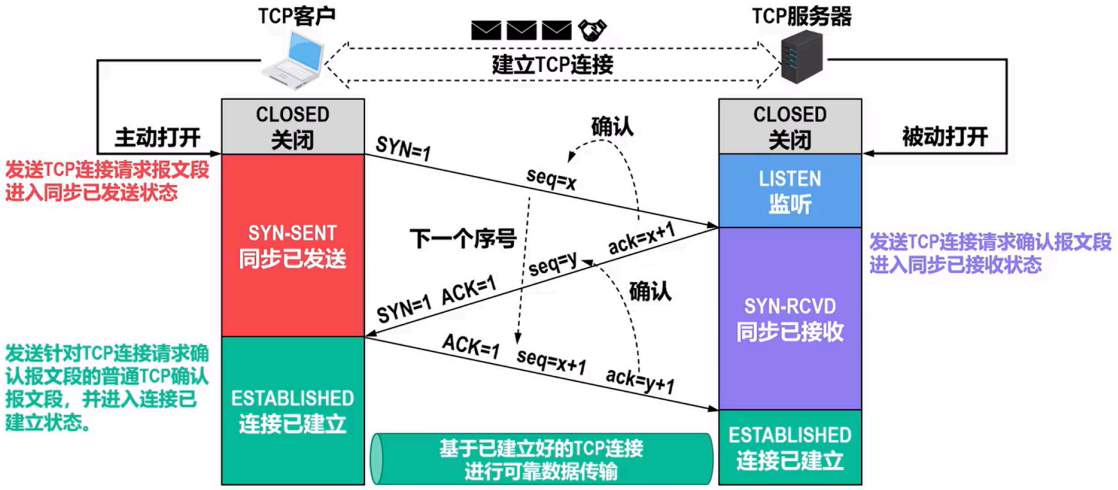
例：第一个字节为201，正确接收了TCP服务器发来序号到799的全部数据，期望收到800以后的数据，ack为1，确认号有效



TCP“三报文握手”建立连接

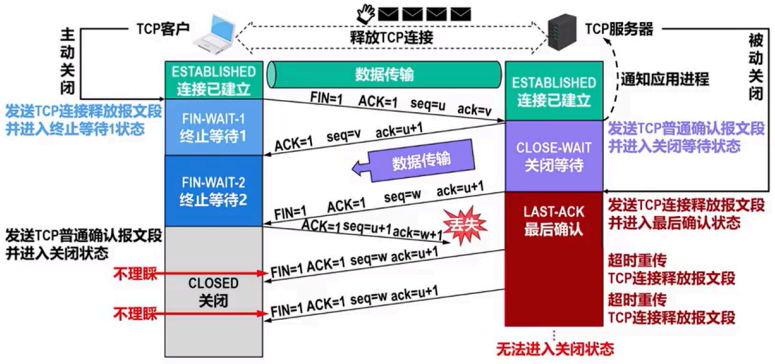
- 使双方能够确知对方的存在
- 使双方能够协商一些参数

- 使双方能够对运输实体资源进行分配和初始化

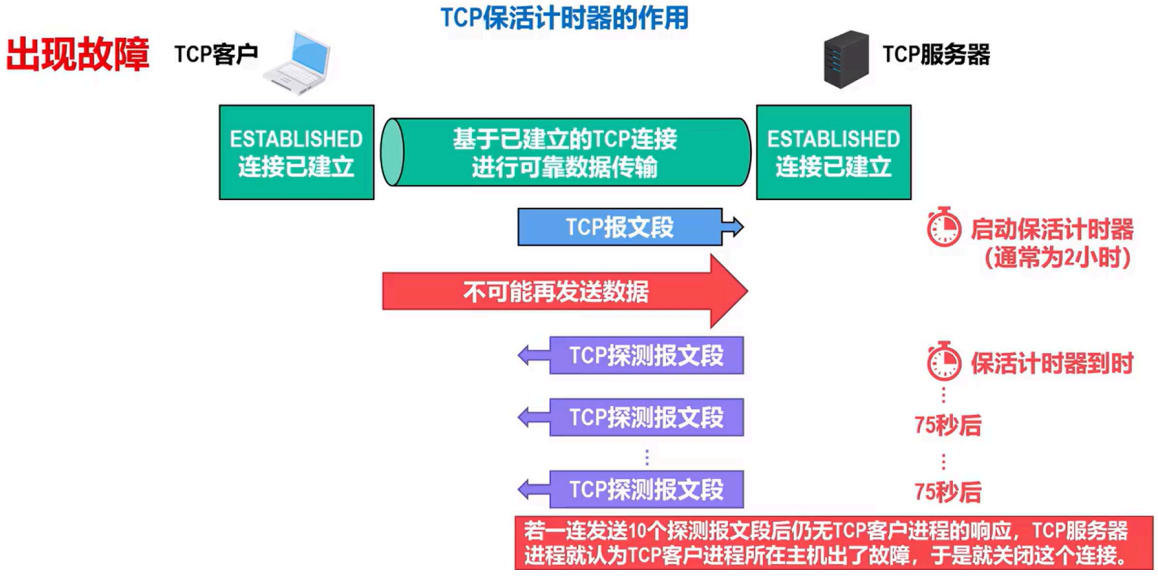


采用“三报文握手”而不是“两报文握手”来建立TCP连接，是为了防止已失效的TCP连接请求报文段突然又传送到了TCP服务器进程，因而导致错误

TCP“四报文挥手”释放连接



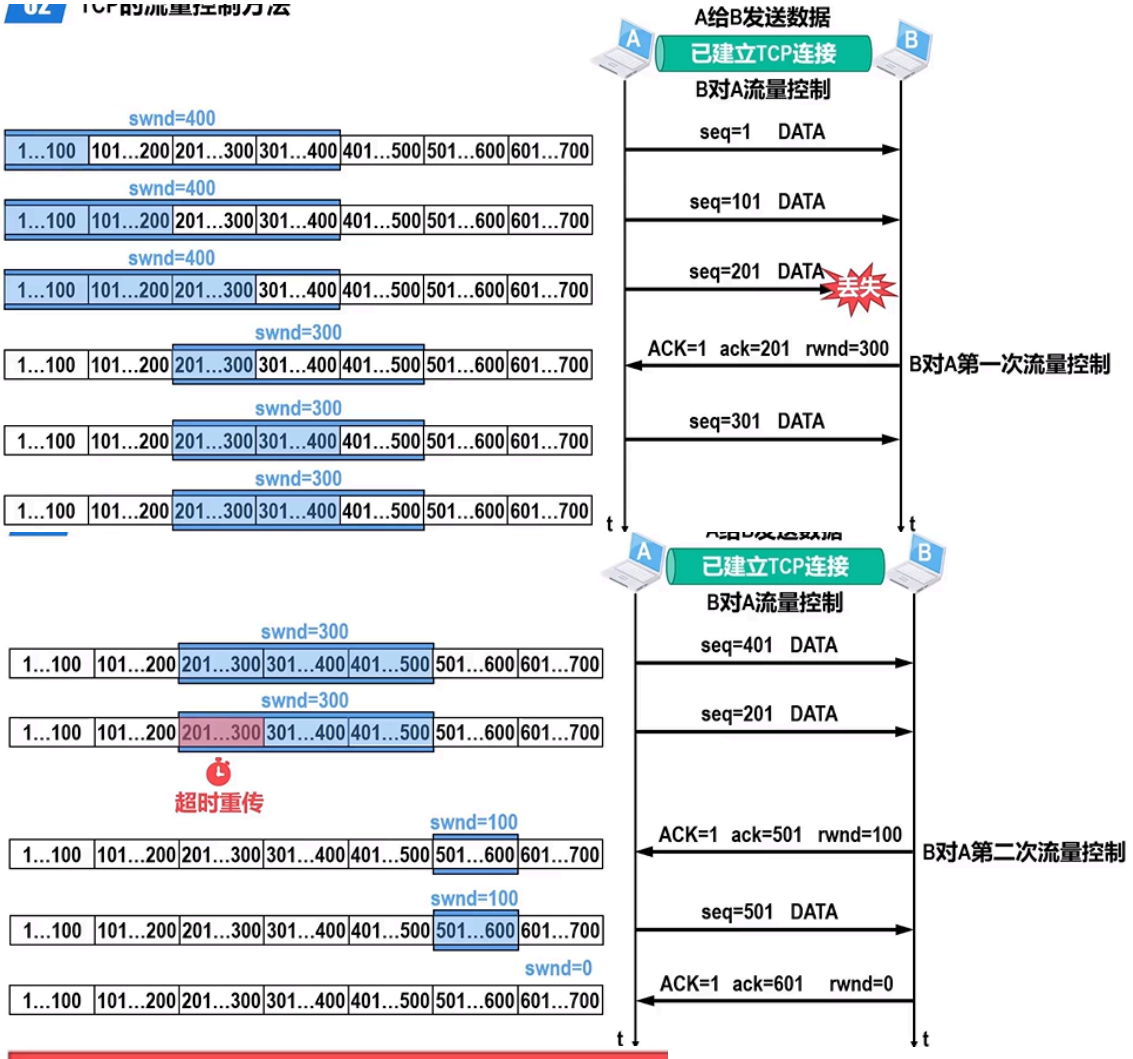
- 处于时间等待（TIME-WAIT）状态后要经过2MSL时长，可以确保TCP服务器进程能够收到最后一个TCP确认报文段而进入关闭（CLOSED）状态。
- 另外，TCP客户进程在发送完最后一个TCP确认报文段后，再经过2MSL时长，就可以使本次连接持续时间内所产生的的所有报文段都从网络中消失。这样就可以使下一个新的TCP连接中不会出现旧连接中的报文段。



TCP的流量控制

- 解决因发送方发送数据太快而导致接收方来不及接收，造成接收方的接收缓存溢出的问题

- 接收方根据自己的接受能力（接收缓存的可用空间大小）控制发送方的发送速率



为了打破由于非零窗口通知报文段丢失而引起的双方互相等待的死锁局面，TCP为每一个连接都设有一个持续计时器。

- 只要TCP连接的一方收到对方的零窗口通知，就启动持续计时器。
- 当持续计时器超时时，就发送一个零窗口探测报文段，仅携带1字节的数据。
- 对方在确认这个零窗口探测报文段时，给出自己现在的接收窗口值。
- 如果接收窗口值仍然是0，那么收到这个报文段的一方就重新启动持续计时器。
- 如果接收窗口值不是0，那么死锁的局面就可以被打破了。

拥塞控制

- 拥塞：在某段时间，若对网络中某一资源的需求超过了该资源所能提供的可用部分，网络性能就要变坏

- 若出现拥塞而不进行控制，整个网络的吞吐量将随输入负荷的增大而下降
- 源点根据各方面因素，按照拥塞控制算法自行控制发送速率

开环控制

- 试图用良好的设计来解决问题。
- 从一开始就保证问题不会发生。
- 一旦系统启动并运行起来，就不需要中途修正。

当网络的流量特征可以准确规定且性能要求可以事先获得时，适合使用开环控制。

闭环控制

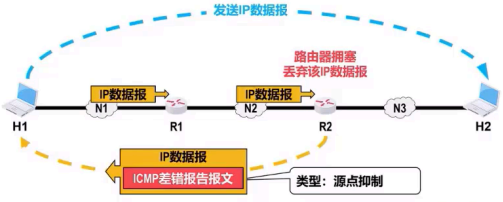
- 基于反馈的控制方法，包括以下三个部分：
  1. 监测网络拥塞在何时、何地发生。
  2. 把拥塞发生的相关信息传送到可以采取行动的地方。
  3. 调整网络的运行以解决拥塞问题。

当网络的流量特征不能准确描述或者当网络不提供资源预留时，适合使用闭环控制。因特网采用的就是闭环控制方法。

根据拥塞信息的反馈形式，可将闭环拥塞控制算法分为

显式反馈算法

从拥塞节点（即路由器）向源点提供关于网络中拥塞状态的显式反馈信息。



隐式反馈算法

源点自身通过对网络行为的观察（例如超时重传或往返时间RTT）来推断网络是否发生了拥塞。TCP采用的就是隐式反馈算法。

拥塞控制并不仅仅是运输层要考虑的问题。显式反馈算法就必须涉及网络层。虽然一些网络体系结构（如ATM网络）主要在网络层实现拥塞控制，但因特网主要利用隐式反馈在运输层实现拥塞控制。

慢开始和拥塞避免

慢开始 (slow-start) | 拥塞避免 (congestion avoidance) | 快重传 (fast retransmit) | 快恢复 (fast recovery)



发送窗口swnd
拥塞窗口cwnd
慢开始门限ssthresh

不考虑流量控制

swnd = cwnd

- cwnd的维护原则：只要网络没有出现拥塞，拥塞窗口就再增大一些，但只要网络出现拥塞，拥塞窗口就减少一些。
- 判断网络出现拥塞的依据：没有按时收到应当到达的TCP确认报文段而产生了超时重传。

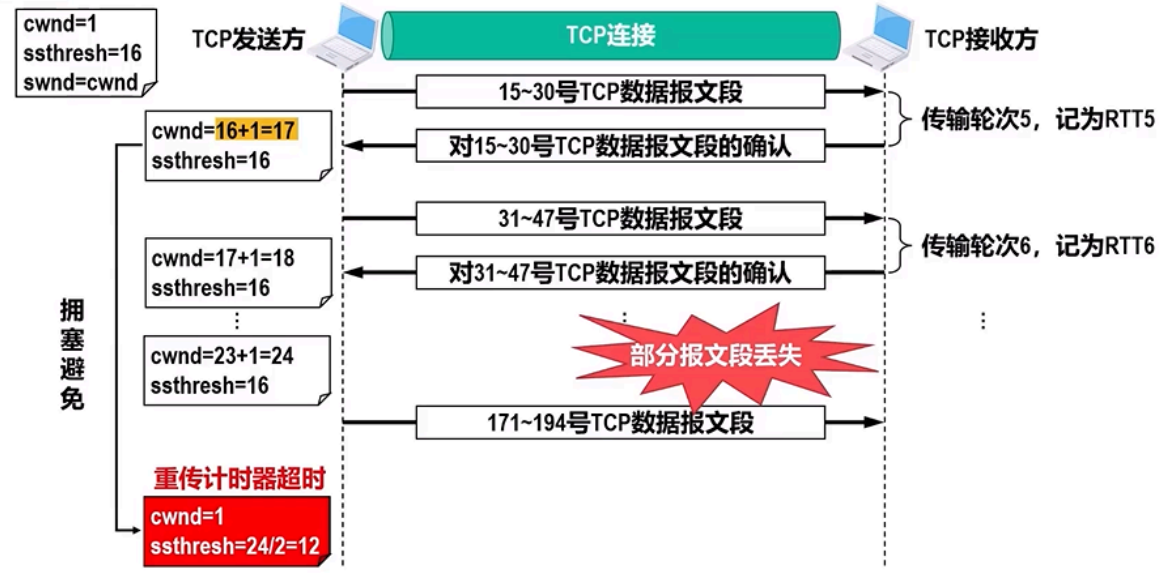
- 当cwnd < ssthresh时，使用慢开始算法。
- 当cwnd > ssthresh时，停止使用慢开始算法而改用拥塞避免算法。
- 当cwnd = ssthresh时，既可使用慢开始算法，也可使用拥塞避免算法。

慢开始

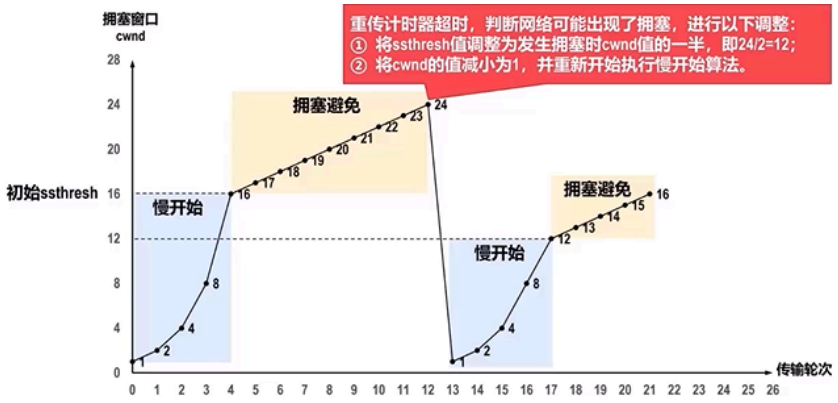


拥塞避免

当部分报文段丢失时，cwnd重置为1，sstresh为丢失时cwnd的一半



总结



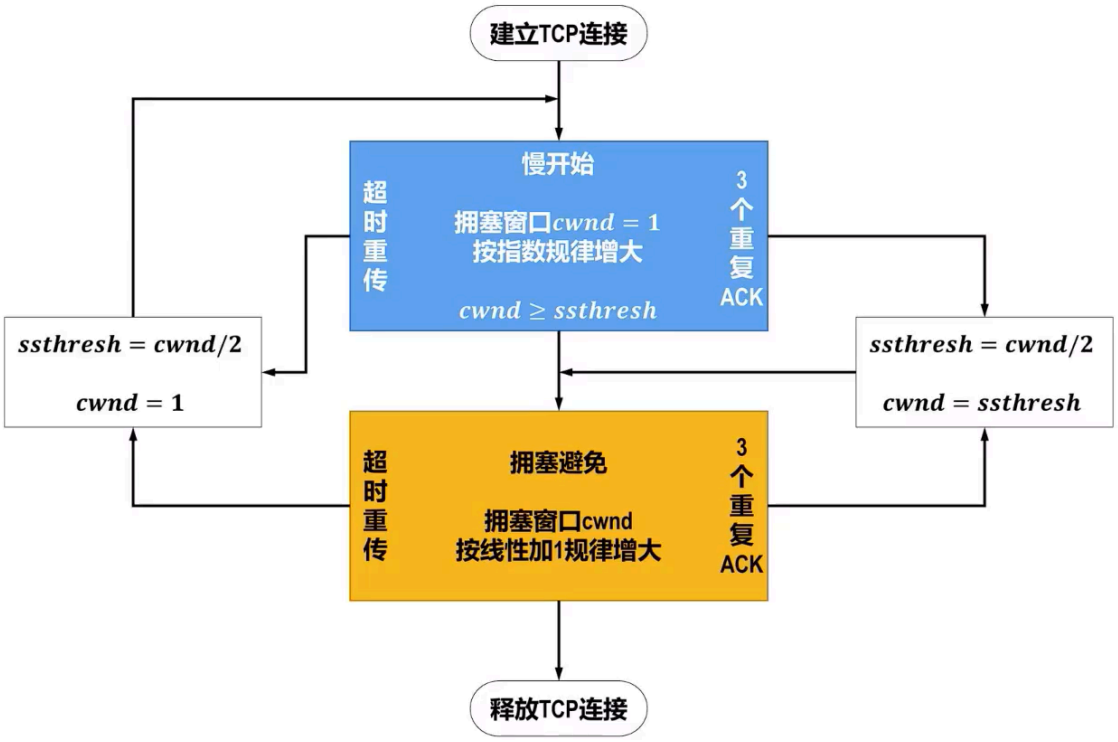
“慢开始”是指一开始向网络注入的报文段少，而并不是指拥塞窗口cwnd的值增长速度慢。

“拥塞避免”也并非指完全能够避免拥塞，而是指在拥塞避免阶段将cwnd值控制为按线性规律增长，使网络比较不容易出现拥塞。

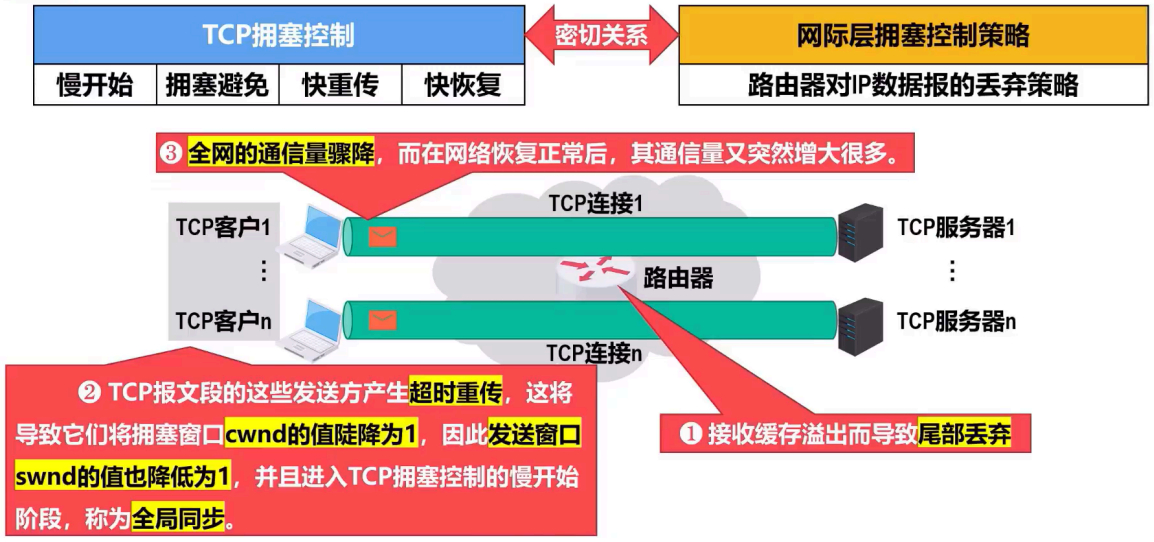
快重传和快恢复

- 快重传可以让发送方尽早知道发生个别TCP报文段的丢失
- 指发送方尽快进行重传，而不是等重传计时器超时再重传
- 接收方要立即发送确认，即使收到了失序的报文段也要立即对已收到的报文段的重复确认
- 发送方一旦收到3个连续的重复确认，九江相应报文段立即重传
- 发送方收到3个重复确认后，执行快恢复
- 将慢开始门限ssthresh的值和拥塞窗口cwnd的值都调整为当前cwnd值的一半，并开始执行拥塞避免算法

总结



TCP拥塞控制与网际层拥塞控制的关系



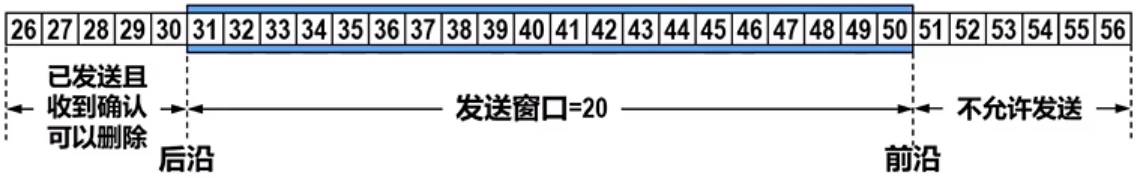
主动队列管理（AQM）：

- 在路由器的队列长度达到某个法制但还未满时就主动丢弃IP数据报

- 当路由器队列长度达到某个值得警惕的数值时，也就是网络出现了某些堵塞征兆时，就主动丢弃到达的IP数据报来造成发送方的超时重传，进而降低发送方的发送速率
  随机早期丢弃（RED）：
- 维护队列长度最小门限和最大门限
  - ☐ **若平均队列长度小于最小门限，则把新到达的IP数据报存入队列进行排队。**
  - ☐ **若平均队列长度大于最大门限，则把新到达的IP数据报丢弃。**
  - ☐ **若平均队列长度在最小门限和最大门限之间，则按照某一丢弃概率p把新到达的IP数据报丢弃（这体现了丢弃IP数据报的随机性）。**

TCP可靠传输的实现

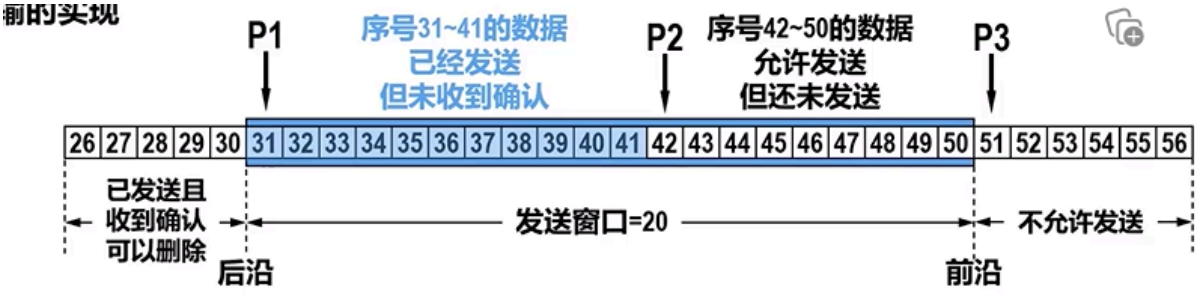
发送窗口的后沿后面是已发送且收到确认可以删除的内容，前沿前面是不允许发送的内容



发送窗口后沿的移动情况：

- 不动（没有收到新的确认）
- 前移 -> （收到新的确认）
  发送窗口前沿的异动情况
- 前移 -> （通常情况）
- 不动
  - 没有收到新的确认，接收方通知的窗口大小也没有改变
  - 收到了新的确认，但接收方通知的窗口缩小了，使得前沿正好不动
- 向后收缩（通知的窗口缩小）

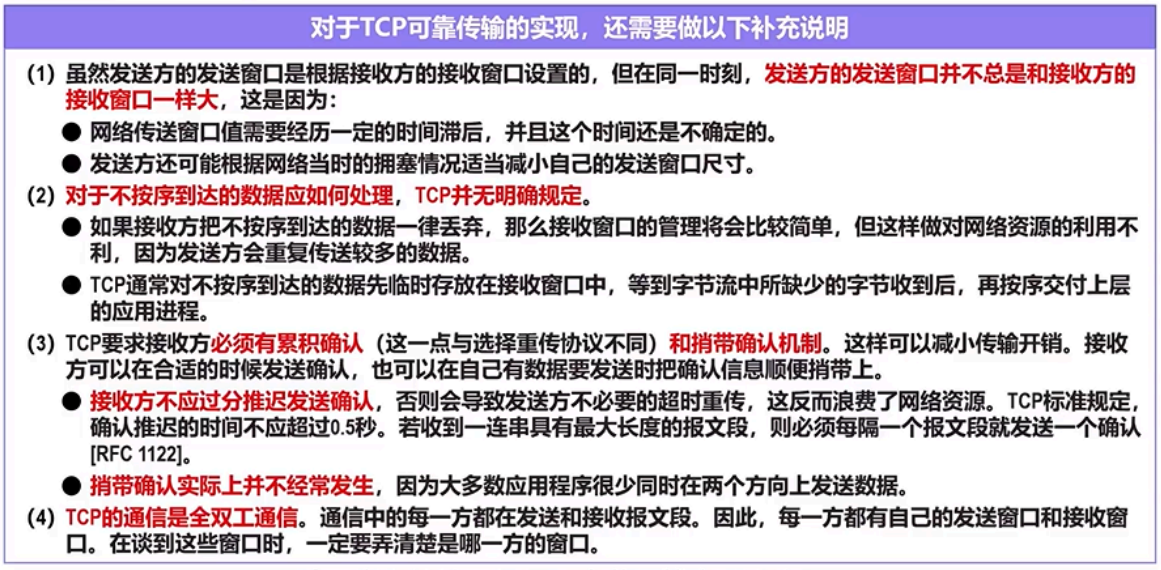
p1指向发送窗口内已发送但还未收到确认的第一个数据的序号
p2指向发送窗口内还未发送的第一个数据的序号
p3指向发送窗口前沿外的第一个数据的序号



这样就可以用P1、P2和P3这三个指针来描述发送窗口的相关信息：

- **小于P1的就是已发送并已收到确认的部分**
- **大于等于P3的就是不允许发送的部分**
- **P3 - P1 = 发送窗口尺寸**
- **P2 - P1 = 已发送但还未收到确认的字节数量**
- **P3 - P2 = 允许发送但当前还未发送的字节数量（又称为可用窗口或有效窗口）**

补充



对于TCP可靠传输的实现，还需要做以下补充说明

(1) 虽然发送方的发送窗口是根据接收方的接收窗口设置的，但在同一时刻，发送方的发送窗口并不总是和接收方的接收窗口一样大，这是因为：

- 网络传送窗口值需要经历一定的时间滞后，并且这个时间还是不确定的。
- 发送方还可能根据网络当时的拥塞情况适当减小自己的发送窗口尺寸。

(2) 对于不按序到达的数据应如何处理，TCP并无明确规定。

- 如果接收方把不按序到达的数据一律丢弃，那么接收窗口的管理将会比较简单，但这样做对网络资源的利用不利，因为发送方会重复传送较多的数据。
- TCP通常对不按序到达的数据先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再按序交付上层的应用进程。

(3) TCP要求接收方必须有累积确认（这一点与选择重传协议不同）和捎带确认机制。这样可以减小传输开销。接收方可以在合适的时候发送确认，也可以在自己有数据要发送时把确认信息顺便捎带上。

- 接收方不应过分推迟发送确认，否则会导致发送方不必要的超时重传，这反而浪费了网络资源。TCP标准规定，确认推迟的时间不应超过0.5秒。若收到一连串具有最大长度的报文段，则必须每隔一个报文段就发送一个确认[RFC 1122]。
- 捎带确认实际上并不经常发生，因为大多数应用程序很少同时在两个方向上发送数据。

(4) TCP的通信是全双工通信。通信中的每一方都在发送和接收报文段。因此，每一方都有自己的发送窗口和接收窗口。在谈到这些窗口时，一定要弄清楚是哪一方的窗口。

TCP超时重传时间的选择

- 不能直接使用略大于某次测量得到的往返时间RTT样本的值作为超时重传时间RTO
- 可以使用加权平均往返时间RTTs

$$RTT_{S1} = RTT_1$$

$$新的RTT_S = (1 - \alpha) \times 旧的RTT_S + \alpha \times 新的RTT样本$$

在上式中，$0 \le \alpha < 1$

若α很接近于0，则新RTT样本对RTTs的影响不大；

若α很接近于1，则新RTT样本对RTTs的影响较大；

已成为建议标准的[RFC 6298]推荐的α值为1/8，即 0.125。

- RTO的值应略大于RTTs

$$RTO = RTT_S + 4 \times RTT_D$$

加权平均往返时间RTT_S

$RTT_{S1} = RTT_1$

新的$RTT_S = (1 - \alpha) \times$ 旧的$RTT_S + \alpha \times$ 新的RTT样本

在上式中，$0 \le \alpha < 1$，已成为建议标准的[RFC 6298]推荐的α值为1/8，即 0.125。

RTT偏差的加权平均RTT_D

$RTT_{D1} = RTT_1 \div 2$

新的$RTT_D = (1 - \beta) \times$ 旧的$RTT_D + \beta \times |RTT_S -$ 新的RTT样本$|$

在上式中，$0 \le \beta < 1$，已成为建议标准的[RFC 6298]推荐的β值为1/4，即 0.25。

通过上述两个例子可以看出：当发送方出现超时重传后，收到确认报文段时是无法判断出该确认到底是对原数据报文段的确认还是对重传数据报文段的确认，也就是无法准确测量出RTT，进而无法正确计算RTO。

Karn算法

在计算加权平均RTTs时，只要报文段重传了，就不采用其RTT样本。换句话说，出现重传时，不重新计算RTTs，进而RTO也不会重新计算。

新问题

设想出现这样的情况：报文段的时延突然增大很多，并且之后很长一段时间都会保持这种时延（这可能是因为网络拓扑发生了变化）。因此在原来得出的RTO内，不会收到确认报文段，于是就重传报文段。但根据Karn算法，不考虑重传的报文段的RTT样本，因此RTO就无法更新，这会导致报文段反复被重传。

修正Karn算法

报文段每重传一次，就把RTO增大一些。典型的做法是将新RTO的值取为旧RTO的2倍。

TCP的选择确认

选择确认来实现只传送缺少的数据而不重传已经正确到达、只是未按序到达的数据

